UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Mini Project Report

On

Puchoo- Quora like Web application

SUBMITTED BY:

ABHENISH KUMAR

UID: 21MCI10087

CLASS: MCA (AI & ML)

SECTION: 24MAM4-4

SUBMITTED TO:

Ms. Shruti Sharma

Assistant Professor

Table of Content

S. No.	Name	Page no.
1	Acknowledgement	3
2	Introduction /Background	4
3	Objective/ Technologies Used	4
4	Technologies Used	4
5	Requirements and System Design	5-6
6	Source Code & Output	7-12
7	Functionalities	13
8	Conclusion/Future Work/Learning Outcomes	14

Acknowledgement

I would like to express my heartfelt gratitude to all those who contributed to the successful development of Puchoo – a dynamic and interactive Q&A platform.

First and foremost, I am thankful to my mentors and instructors for their continuous guidance and encouragement throughout this project. Their insights into web development and system design have been invaluable in shaping the technical direction of this work.

I would also like to acknowledge my peers and friends for their constructive feedback and support during the testing and debugging phases. Their suggestions helped improve the user experience and functionality of the platform.

Lastly, I am grateful for the open-source community and the developers behind the technologies used in this project. Tools like Node.js, MySQL, Bootstrap, and jQuery have made it possible to bring this idea to life with both functionality and creativity.

Puchoo stands as a testament to collaborative learning, curiosity, and the power of innovation, and I am proud to present it as a step forward in building engaging educational tools for the digital world.

1. Introduction

This report outlines the development of "Puchoo," a vibrant question-and-answer platform inspired by the spirit of curiosity and community engagement. Designed as a modern alternative to existing Q&A systems, Puchoo offers users a dynamic space to post questions, share answers, and interact through likes and dislikes, all wrapped in an eye-catching interface. The project showcases a blend of technical innovation and aesthetic appeal, culminating in a fully functional web application with a unique identity.

2. Background

Puchoo emerges from the desire to create an accessible, customizable Q&A platform that fosters knowledge sharing and social interaction. Drawing inspiration from popular platforms like Quora, it reimagines the experience with a focus on education and community, hinted at by the playful name "Puchoo," derived from the Hindi term for asking. The project leverages a client-server architecture, integrating a MySQL database for data persistence and a visually engaging design to enhance user experience, making it suitable for learners, professionals, and enthusiasts alike.

3. Objectives

- Develop a comprehensive Q&A web application with seamless user authentication.
- Create an attractive user interface featuring a colorful animated background to captivate users.
- Implement core features including question posting, answering, liking, disliking, and a logout functionality.
- Design a scalable and maintainable system using contemporary web technologies.
- Ensure an intuitive and engaging user experience that encourages active participation.

4. Technologies Used

- Frontend: HTML5 for structural foundation, CSS3 with advanced animations for a stunning gradient background, and JavaScript with jQuery for dynamic client-side interactions.
- Backend: Node.js with Express.js to manage server-side logic and API endpoints.
- Database: MySQL2 for robust storage of questions, answers, and user engagement metrics.
- Session Management: Express-session to handle user authentication and session persistence.

- Middleware: CORS and body-parser to facilitate cross-origin requests and JSON data processing.
- External Libraries: Bootstrap 4.5.2 for responsive and professional design elements, and CDN-hosted jQuery 3.5.1 for efficient DOM manipulation

5. Requirements

- Node.js and npm for development environment setup.
- A MySQL server with a dedicated quora_clone database.
- A modern web browser (e.g., Chrome, Firefox) for optimal experience.
- A code editor (e.g., VS Code) and terminal for project management.

6. System Design

- **Architecture Overview**

  Puchoo follows a client-server architecture, where the frontend (HTML, CSS, jQuery) communicates with the backend (Node.js + Express.js) through RESTful APIs. The system relies on a MySQL relational database to store user data, questions, answers, and user interactions (likes/dislikes). The application supports user sessions for login/logout mechanisms, ensuring security and personalized user experience.

- **Component Overview**
- **Frontend (Client Side)**

  - Single Page Layout: The UI is designed as a single-page interface, enabling smooth transitions without full page reloads.
  - Animated Gradient Background: Visually appealing CSS-based animations to increase user engagement.
  - Responsive Design: Uses Bootstrap 4.5.2 for seamless display across devices.
  - User Interactions: jQuery handles DOM manipulation, AJAX requests, and client-side validations.

- **Backend (Server Side)**

  - Express.js server handles API routing and business logic.
  - Node.js manages runtime environment for server-side scripting.

- Express-session manages user authentication and session data.
- Body-parser parses incoming JSON requests.
- CORS handles cross-origin requests if needed.

- **Database (MySQL)**

  - Stores questions, answers, users, and interactions in a structured schema.
  - Ensures referential integrity between questions and answers via foreign keys.
  - **Database Schema:**
    - questions table: id (primary key), question (text), category (varchar), username (varchar), likes (integer), dislikes (integer).
    - answers table: id (primary key), question_id (foreign key), answer (text), username (varchar).

Data Flow

```
[Start: User Access]
        ↓
[http://localhost:3000/]
        ↓
  [Authentication]
        ↓
   ┌──────────┐
   ↓          ↓
[/login]    [/session]
(Session Tracking)
        ↓
 [CRUD Operations]
        ↓
   ┌──────┬──────────┬──────────┐
   ↓      ↓          ↓          ↓
[/question] [/answer] [/questions] [/answers/:qid]
        ↓
 [Engagement Features]
        ↓
```

------→

```
[Engagement Features]
        ↓
   ┌──────────┐
   ↓          ↓
[/like/:qid] [/dislike/:qid]
        ↓
     [Logout]
        ↓
    [/logout]
        ↓
[End: Session Terminated]
```

7. Source Code & Output

Backend code:-

```
const express = require('express');

const session = require('express-session');

const bodyParser = require('body-parser');

const cors = require('cors');

const mysql = require('mysql2');

const app = express();

const path = require('path');

app.use(express.static(path.join(__dirname, 'public')));

app.get('/', (req, res) => {

  res.sendFile(path.join(__dirname, 'public', 'index.html'));

});

app.use(cors({

  origin: 'http://localhost:3000', // Match the server's own origin

  credentials: true

}));app.use(bodyParser.json());

app.use(session({

  secret: 'quoraclone-secret',

  resave: false,

  saveUninitialized: false, // Avoid unnecessary sessions

  cookie: {

    secure: false, // Set to true if using HTTPS

    maxAge: 24 * 60 * 60 * 1000 // Session expires after 24 hours

  }
```

```
  const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'Root123',
  database: 'quora_clone'
});
db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});app.post('/login', (req, res) => {
  const { username } = req.body;
  if (username) {
    req.session.username = username;
    res.json({ username });
  } else {
    res.status(400).json({ error: 'Username is required' });
  }
});
app.get('/session', (req, res) => {
  console.log('Session check:', req.session); // Debug log
  if (req.session.username) {
    res.json({ username: req.session.username });
  } else {
    res.json({ username: null });
  }
```

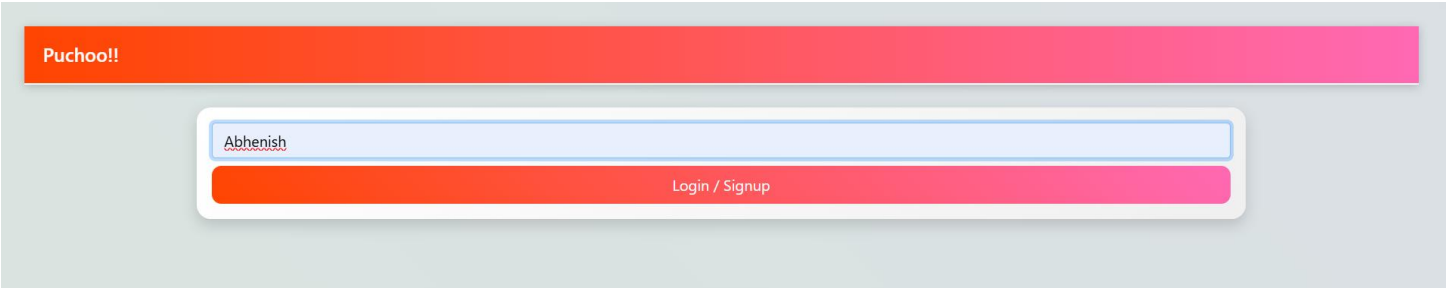
```
app.post('/question', (req, res) => {
  const { question, category } = req.body;
  const username = req.session.username;
  if (username && question && category) {
    const sql = 'INSERT INTO questions (question, category, username) VALUES (?, ?, ?)';
    db.query(sql, [question, category, username], (err) => {
      if (err) {
        console.error('Error inserting question:', err);
        res.status(500).send('Error posting question');
      } else {
        res.sendStatus(200);
      }
    });
  } else {
    res.status(400).send('Missing required fields');
  }
});

app.get('/questions', (req, res) => {
  const sql = 'SELECT * FROM questions ORDER BY id DESC';
  db.query(sql, (err, results) => {
    if (err) {
      console.error('Error fetching questions:', err);
      res.status(500).send('Error fetching questions');
```

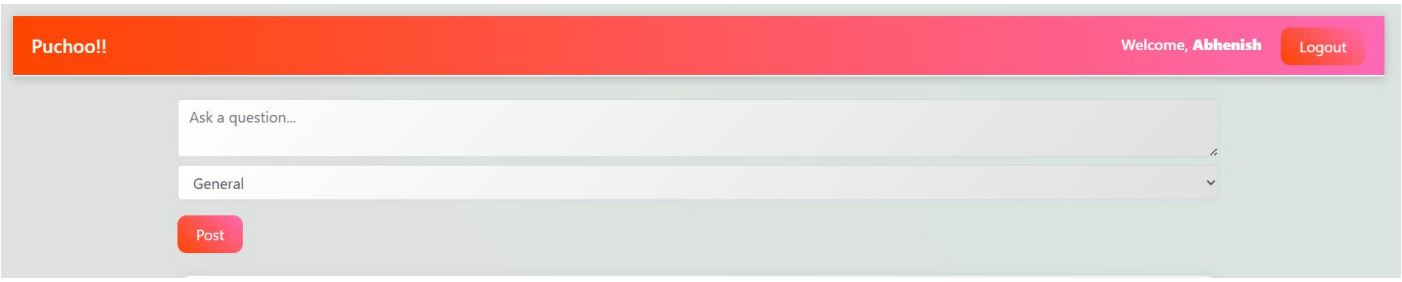
```
    } else {
      res.sendStatus(200);
    }
  });
});


app.post('/dislike/:qid', (req, res) => {
  const qid = req.params.qid;
  const sql = 'UPDATE questions SET dislikes = dislikes + 1 WHERE id = ?';
  db.query(sql, [qid], (err) => {
    if (err)
app.post('/logout', (req, res) => {
  console.log('Logout attempt, session before:', req.session); // Debug log
  req.session.destroy(err => {
    if (err) {
      console.error('Error destroying session:', err);
      res.status(500).send('Error logging out');
    } else {
      console.log('Session destroyed, session after:', req.session); // Debug log
      res.sendStatus(200);
    }
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

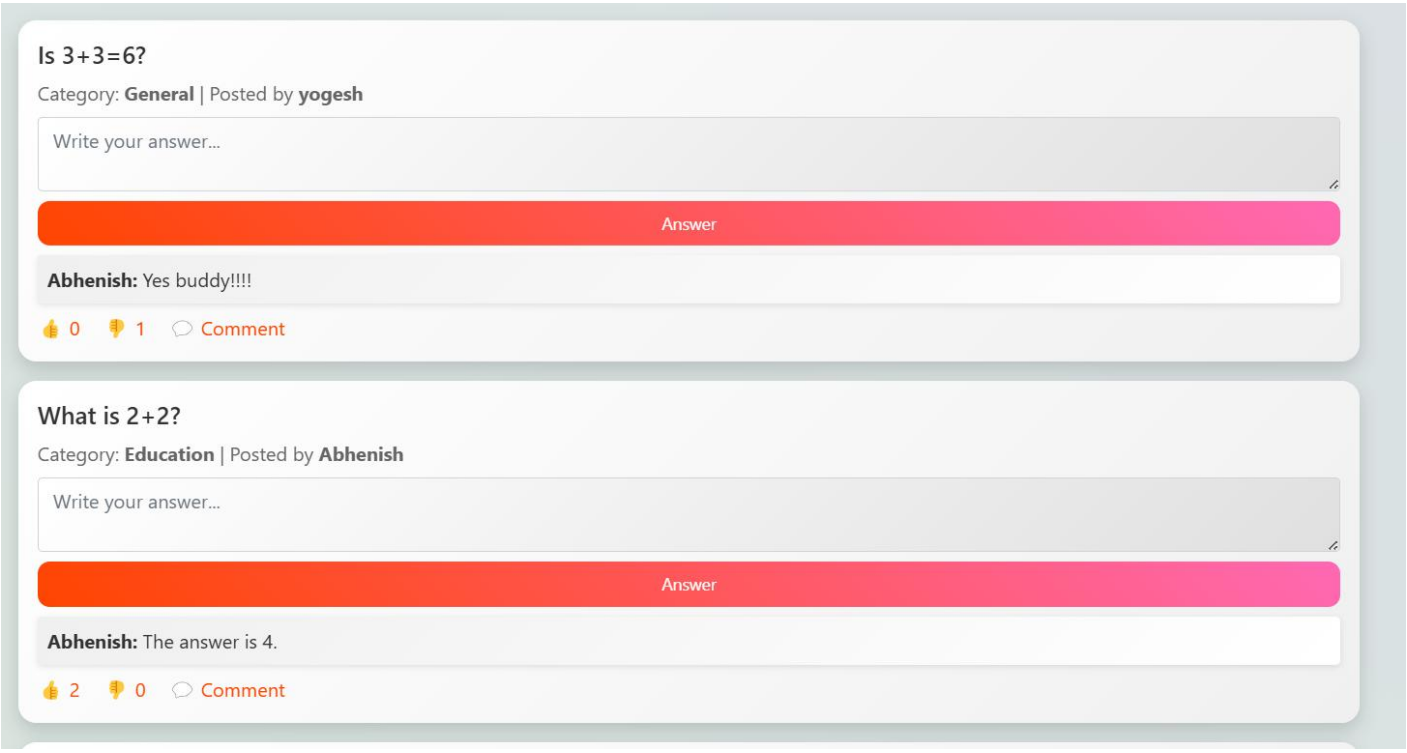
Front-End code:-

```
<body>
 <div class="content-overlay">
  <nav class="navbar navbar-expand-lg">
   <a class="navbar-brand" href="#">Puchoo!!</a>
   <div class="ml-auto" id="userArea"></div>
  </nav>


  <div class="container mt-4">
   <div id="authArea"></div>
   <div id="qaSection" style="display:none">
    <div class="mb-4">
     <textarea class="form-control" id="questionInput" placeholder="Ask a question..."></textarea>
     <select id="category" class="form-control my-2">
      <option value="General">General</option>
      <option value="Science">Science</option>
      <option value="Technology">Technology</option>
      <option value="Education">Education</option>
     </select>
     <button class="btn btn-danger mt-2" onclick="postQuestion()">Post</button>
    </div>
    <div id="questionList"></div>
   </div>
  </div>
```

Output


Puchoo!!
Abhenish
Login / Signup


Puchoo!!
Welcome, Abhenish
Logout
Ask a question...
General
Post


Is 3+3=6?
Category: General | Posted by yogesh
Write your answer...
Answer
Abhenish: Yes buddy!!!!
0 1 Comment
What is 2+2?
Category: Education | Posted by Abhenish
Write your answer...
Answer
Abhenish: The answer is 4.
2 0 Comment

8. Functionalities

User Authentication

- Signup/Login system with session tracking.
- Session-based access control: Only logged-in users can post, answer, like/dislike.

Ask a Question

- Authenticated users can submit a question with optional category tags.
- Stored in the questions table with an initial like/dislike count of 0.

Post an Answer

- Users can reply to any question.
- Stored in the answers table, linked to the corresponding question by question_id.

Like/Dislike System

- Logged-in users can like or dislike a question.
- Updates likes or dislikes column in the questions table.
- Optionally, prevent multiple votes by the same user per question (can be enhanced with a separate vote tracking table).

View Questions and Answers

- All users (logged-in or not) can view:
  - A list of all questions.
  - Answers under each question.
  - Like/dislike counts.

Logout

- Ends the session and redirects to the home or login page.

9. Conclusion

Puchoo stands as a successful implementation of a Q&A platform, delivering on its promise of an engaging and functional user experience. The animated background enhances visual appeal, while features like login, logout, and interactive engagement options showcase its versatility. Built with a robust technology stack, Puchoo demonstrates a blend of creativity and technical prowess, establishing a foundation for a thriving community-driven application.

10. Future Work

- Enhanced Session Management: Integrate a persistent session store to maintain user sessions across server restarts.
- User Profiles: Develop personalized profile pages showcasing users' question and answer history.
- Real-Time Features: Incorporate WebSockets for live updates on questions and answers.
- Mobile Optimization: Refine the CSS for a fully responsive design across mobile devices.
- Advanced Security: Implement input sanitization, password protection, and HTTPS support for a secure environment.

11. Learning Outcomes:

- Proficiency in building a robust backend using Node.js and Express.js for server-side logic.
- Mastery of session management with Express-session for user authentication and logout functionality.
- Skill in designing and implementing RESTful APIs for CRUD operations (e.g., /question, /answer, /questions).
- Expertise in integrating MySQL database with the server for persistent data storage and retrieval.

12. GitHub link:

https://github.com/abhenish/AIP

13. Bibliography:

- • Node.js. (n.d.). *Node.js*. Retrieved from https://nodejs.org/
- • Express.js. (n.d.). *Fast, unopinionated, minimalist web framework for Node.js*. Retrieved from https://expressjs.com/
- • MySQL. (n.d.). *The world's most popular open source database*. Retrieved from https://www.mysql.com/